Estrutura de Dados Lineares

Introdução à Análise da Complexidade de Algoritmos

Análise de algoritmos: ideia geral

- Em ciência da computação, a análise de algoritmos é a área de pesquisa cujo foco são os algoritmos.
- A análise de algoritmos busca responder a seguinte pergunta:

podemos fazer um algoritmo mais eficiente?

Análise de algoritmos: ideia geral

- Múltiplas soluções, eficiências diferentes
- Em programação, podemos resolver um problema de várias maneiras diferentes, isto é, podemos utilizar algoritmos diferentes para um mesmo problema.

- **Algoritmos diferentes, mas capazes de resolver o mesmo problema não necessariamente o fazem com a mesma eficiência.**

Análise de algoritmos: ideia geral

- **Diferenças de eficiência**
- Podem ser:
  - irrelevantes para um pequeno número de elementos processados;
  - **crescer proporcionalmente** com o número de elementos processados.

- **Para comparar a eficiência dos algoritmos foi criada uma medida chamada complexidade computacional.**

Análise de algoritmos: ideia geral

- A complexidade computacional indica o **custo** ao se aplicar um algoritmo:

custo = memória + tempo

- **memória**: quanto de espaço o algoritmo consome.
- **tempo**: duração da execução.

Análise de algoritmos: ideia geral

- **Duas abordagens**

- **Análise empírica**: comparação entre os programas.

- **Análise matemática**: estudo das propriedades do algoritmo.

Análise empírica

- Avalia o custo (ou complexidade) **pela execução do programa** quando implementado.

- **Na análise empírica, um algoritmo é analisado pela execução de seu programa correspondente.**

Análise empírica

- **Vantagens**
- Avaliar desempenho em uma configuração específica (computador/linguagem).
- Considerar custos não aparentes (ex.: **alocação de memória**).
- Comparar **computadores** e **linguagens**.

Análise empírica

- **Dificuldades**
- Exige implementar (depende da habilidade do programador).
- Pode ser mascarada por **hardware/software** do momento.
- Natureza dos dados: reais, aleatórios ou perversos.

- **Dados aleatórios** → desempenho **médio**.
- **Dados perversos** → **pior caso**.

Análise matemática

- Medir tempo de forma **independente** do hardware e da linguagem.
- **Estuda a ideia do algoritmo ao nível conceitual**, usando um “computador idealizado” e mantendo **apenas os custos dominantes**.
- Ignora detalhes de baixo nível (linguagem, hardware, instruções da CPU).
- Permite entender como o custo cresce com o tamanho da entrada.

Análise matemática

- **Contando instruções**
- **Instrução simples**: executada diretamente pela CPU (ou muito perto disso).
- Tipos: atribuição; acesso a elemento do array; comparação; incremento; operações aritméticas básicas.
- Assumiremos custo **igual** para instruções simples.
- **if** tem **custo 0** (não conta) nesta análise (nosso exemplo).

Análise matemática

- **Exemplo**

```
int M = A[0];

for(i = 0; i < n; i++){
        if (A[i] >= M) {
                M = A[i];
        }
}
```

Análise matemática

- **Exemplo**
- **Linha 1**: custo = **1** (acessar A[0] e atribuir).

```
int M = A[0];

for(i = 0; i < n; i++){
    if (A[i] >= M) {
        M = A[i];
    }
}
```

Análise matemática

- **Exemplo**
- **Linha 1**: custo = **1** (acessar A[0] e atribuir).
- **Inicialização do for**: custo = **2** (i=0 e primeira comparação i<n).

```
int M = A[0];

for(i = 0; i < n; i++){
    if (A[i] >= M) {
        M = A[i];
    }
}
```

Análise matemática

- **Exemplo**
- **Linha 1**: custo = **1** (acessar A[0] e atribuir).
- **Inicialização do for**: custo = **2** (i=0 e primeira comparação i<n).
- **Custo por iteração**: **2** (i++ e comparação i<n).

```
int M = A[0];

for(i = 0; i < n; i++){
        if (A[i] >= M) {
                M = A[i];
        }
}
```

Análise matemática

- **Exemplo**
- **Linha 1**: custo = **1** (acessar A[0] e atribuir).
- **Inicialização do for**: custo = **2** (i=0 e primeira comparação i<n).
- **Custo por iteração**: **2** (i++ e comparação i<n).
- **Laço vazio** ⇒ f(n) = **2n + 3** (3 antes + 2 por iteração).

```
int M = A[0];

for(i = 0; i < n; i++){
    if (A[i] >= M) {
        M = A[i];
    }
}
```

Análise matemática

- **Custo dominante / pior caso**

- Dentro do for, o if pode ou não executar a atribuição M = A[i].

- Depende do **conteúdo** do array:
  - A1 = {1,2,3,4} → if sempre **verdadeiro**.
  - A2 = {4,3,2,1} → if sempre **falso**.

Análise matemática

- **Em geral, consideramos o pior caso**: quando o maior número de instruções é executado.

- Aqui, **pior caso**: array **crescente** → sempre compara **e** atribui.
  - A1 = {1,2,3,4} → if sempre **verdadeiro**.

- Função custo no pior caso:
- **f(n) = 4n + 3**.

```
int M = A[0];

for(i = 0; i < n; i++){
    if (A[i] >= M) {
        M = A[i];
    }
}
```

Análise matemática: Comportamento assintótico

- Podemos descartar termos “lentos” quando **n** cresce muito.

- Em **f(n) = 4n + 3**:
  - descarta **3** (constante) e **4** (constante multiplicativa) → **f(n) ~ n**.

- Ignoramos essas **constantes**, mantendo a **ideia** do algoritmo.

- **Comportamento assintótico**: o que acontece com f(n) quando

$$n \to \infty$$

Análise matemática: Comportamento assintótico

- Exemplo comparando:

$$g(n) = 1000n + 500$$

$$h(n) = n^2 + n + 1$$

- Para **n** suficientemente grande, **h(n)** domina **g(n)**.

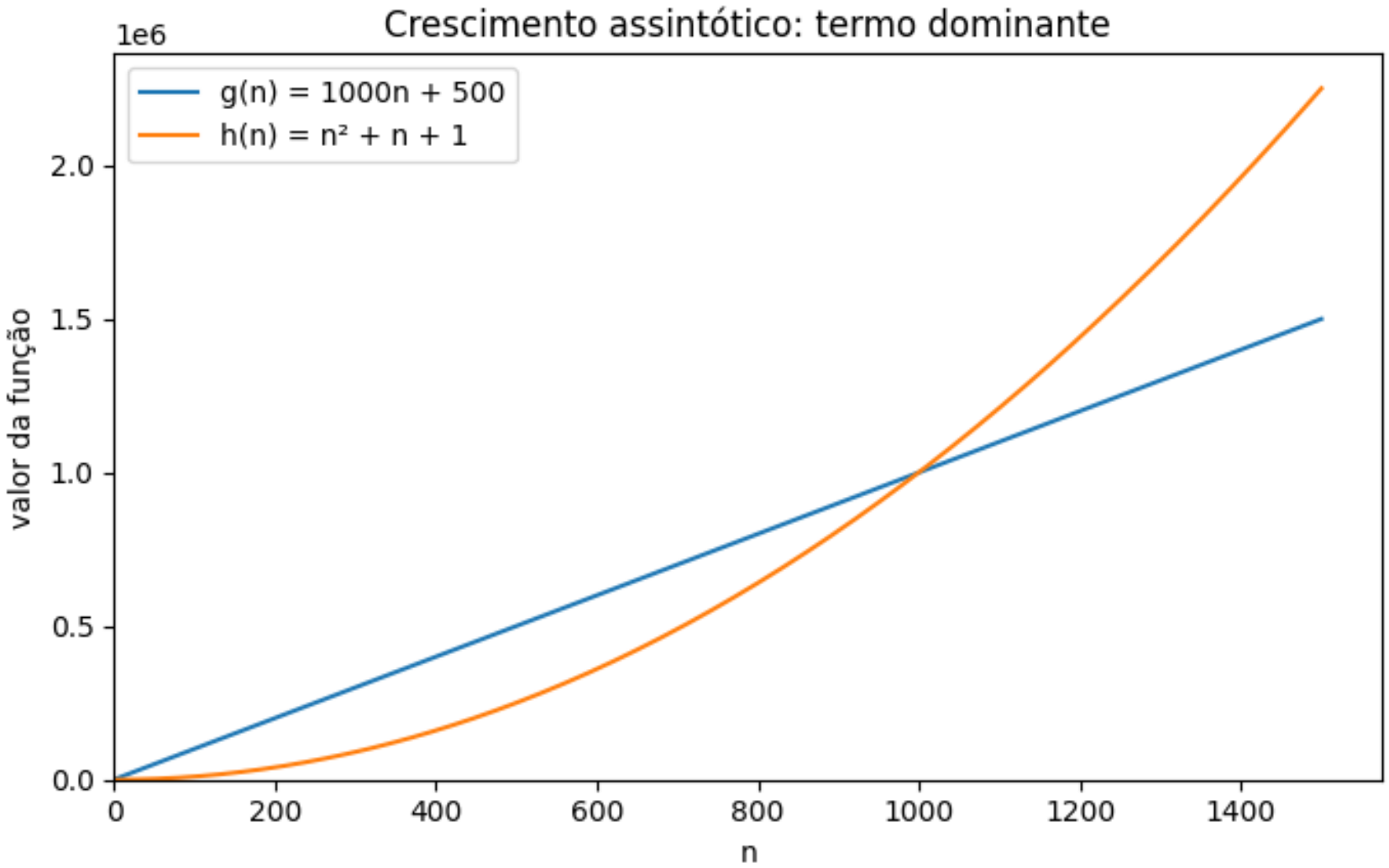
Análise matemática: Comportamento assintótico

- Exemplo comparando:

$g(n) = 1000n + 500$

$h(n) = n^2 + n + 1$

- Para **n** suficientemente grande, **h(n)** domina **g(n)**.



Análise matemática: Comportamento assintótico

- Exemplos de custo e comportamento

Função custo	**Comportamento assintótico**
$f(n) = 105$	1
$f(n) = 15n + 2$	n
$f(n) = n^2 + 5n + 2$	n^2
$f(n) = 5n^3 + 200n^2 + 112$	n^3

- Regra prática:
  - sem laços ⇒ **f(n)=1**;
  - 1 laço até n ⇒ **n**;
  - 2 laços aninhados ⇒ $\mathbf{n^2}$; ...

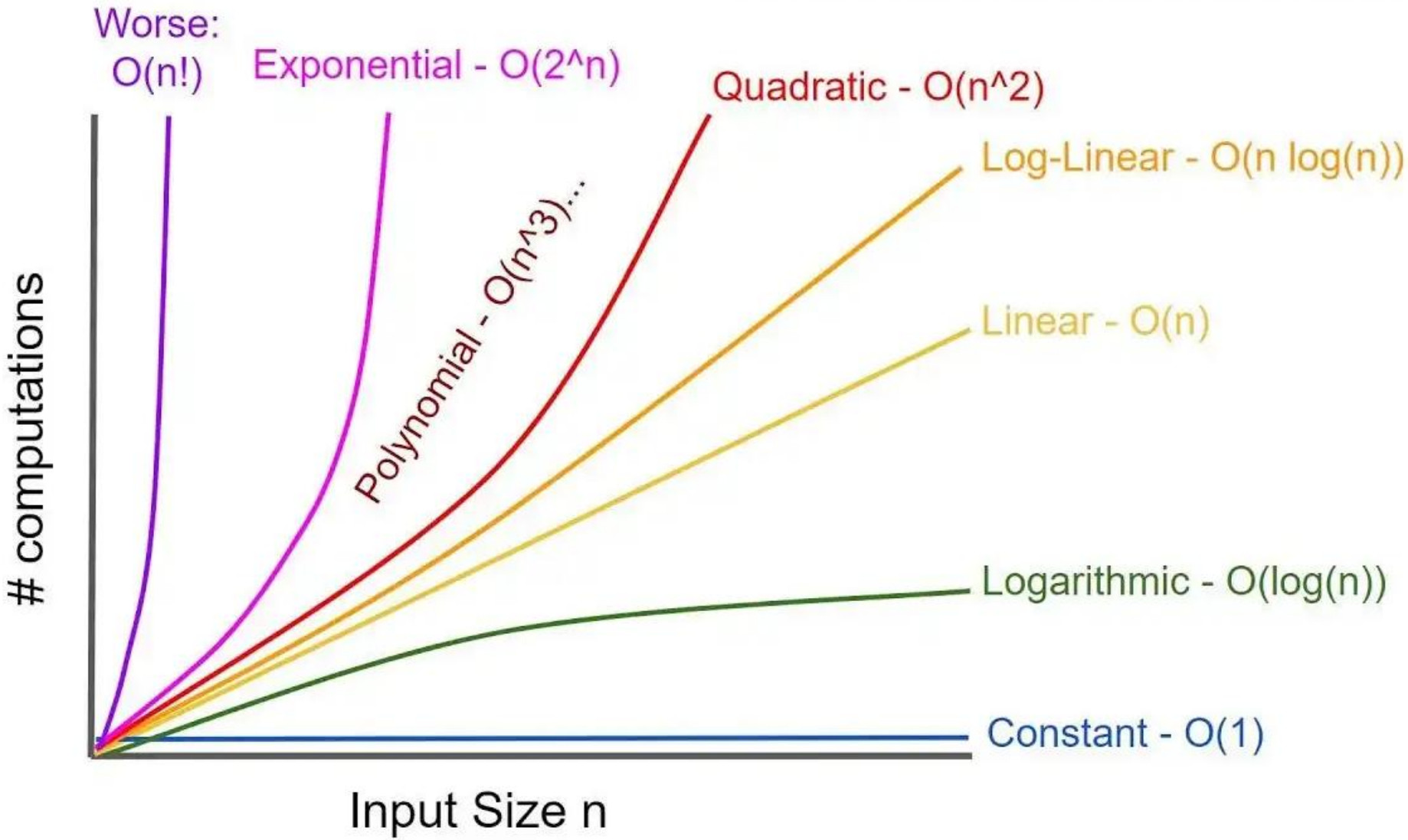
A notação Grande-O: Big-O

- Dentre as várias formas de análise assintótica, a mais conhecida e utilizada é a notação **Grande-O (Big-O)**.

- Representa o **limite assintótico superior** do algoritmo: o custo **no pior caso** para entradas de tamanho *n*.

- **Big-O (O)** descreve o **pior caso**. O comportamento do algoritmo **não ultrapassa** esse limite.

A notação Grande-O: Big-O



A notação Grande-Ω (ômega)

- **Ω** descreve o **limite assintótico inferior** (costuma ser associado ao **melhor caso**).

- **$\Omega(n^2)$** diz que o custo é, assintoticamente, **$\geq n^2$**.

Classes de problemas

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(N!)$$

- **O(1):** ordem constante, ou seja, as instruções são executadas um número fixo de vezes. Não depende do tamanho dos dados de entrada.
- **O(log n):** ordem logarítmica, típica de algoritmos que resolvem um problema transformando-o em problemas menores.
- **O(n):** ordem linear. Em geral, uma certa quantidade de operações é realizada sobre cada um dos elementos de entrada.
- **O(n log n):** ordem log linear. Típica de algoritmos que trabalham com particionamento dos dados. Esses algoritmos resolvem um problema transformando-o em problemas menores, que são resolvidos de forma independente e depois unidos.

Classes de problemas

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(N!)$$

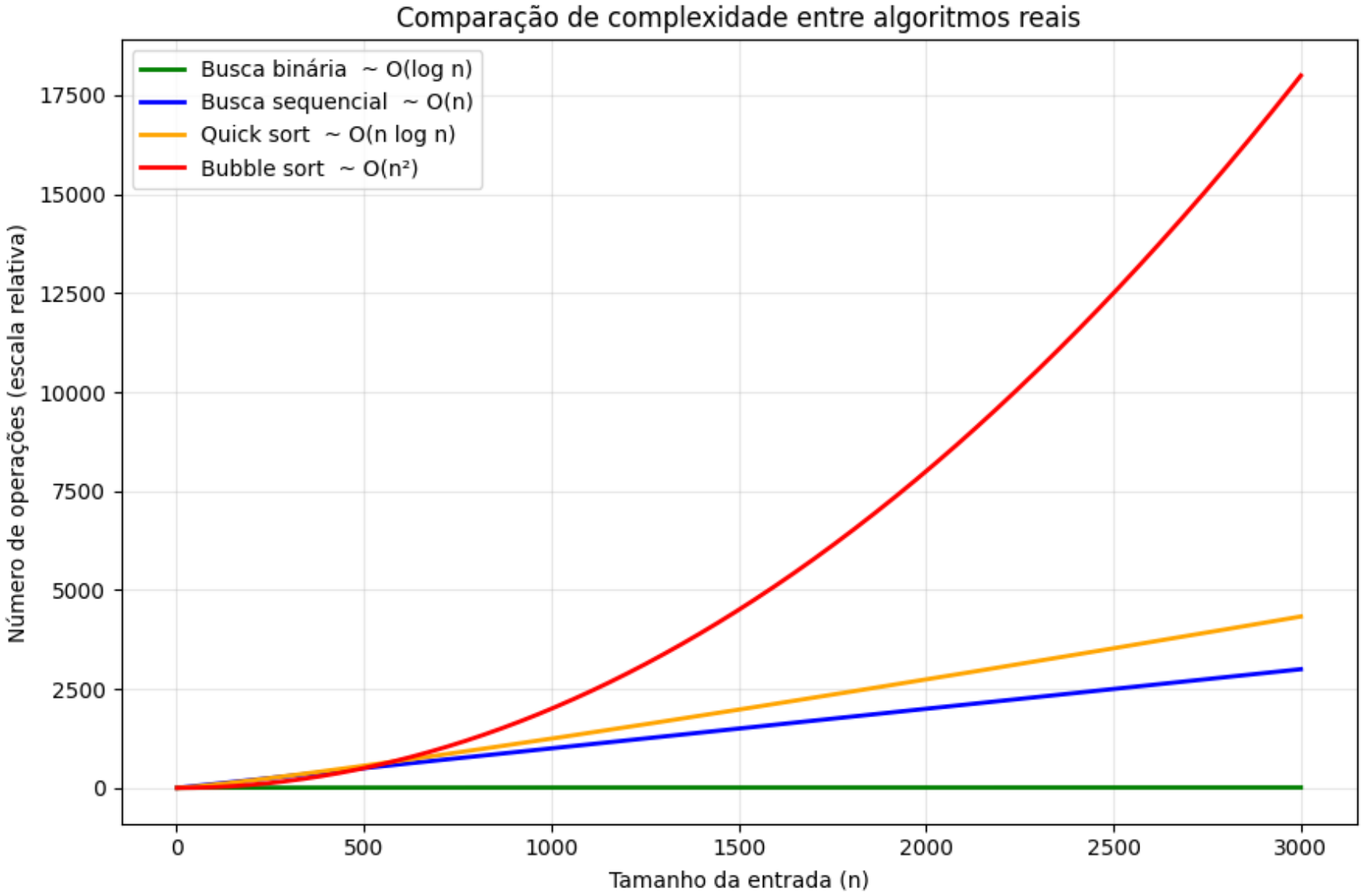
- **$O(n^2)$:** ordem quadrática. Normalmente, ocorre quando os dados são processados aos pares. Uma característica desse tipo de algoritmos é a presença de um aninhamento de dois comandos de repetição.
- **$O(n^3)$:** ordem cúbica. É caracterizada pela presença de três estruturas de repetição aninhadas.
- **$O(2^n)$:** ordem exponencial. Geralmente ocorre quando se usa uma solução de força bruta. Não são úteis do ponto de vista prático.
- **$O(N!)$:** ordem fatorial. Geralmente ocorre quando se usa uma solução de força bruta. Não são úteis do ponto de vista prático. Possui um comportamento muito pior que o exponencial.

Tempo de Execução

f(n)	**n = 10**	**n = 20**	**n = 30**	**n = 50**	**n = 100**
n	1,0E–05 s	2,0E–05 s	4,0E–05 s	5,0E–05 s	6,0E–05 s
n log n	3,3E–05 s	8,6E–05 s	2,1E–04 s	2,8E–04 s	3,5E–04 s
n^2	1,0E–04 s	4,0E–04 s	1,6E–03 s	2,5E–03 s	3,6E–03 s
n^3	1,0E–03 s	8,0E–03 s	6,4E–02 s	0,13 s	1,0 s
2^n	1,0E–03 s	1,0 s	2,8 dias	35,7 anos	365,6 séc.
3^n	5,9E–02 s	58,1 min	3855,2 séc.	2,3E+08 séc.	1,3E+13 séc.

A Tabela mostra uma comparação no tempo de execução de algoritmos que possuem diferentes complexidades. Nesse caso, estamos considerando que nosso computador será capaz de executar 1 milhão de operações por segundo.

Comparação de algoritmos



Análise de Complexidade nas Estruturas de Dados

- Aplicar a **notação Big-O** para analisar as **operações básicas** das estruturas dinâmicas que implementadas em C.

- Cada estrutura possui métodos (funções) que executam em diferentes tempos, dependendo de como a lista, pilha ou fila é organizada internamente.

Análise de Complexidade nas Estruturas de Dados

- **Objetivo**
- Avaliar a **eficiência assintótica** dos principais métodos:
  - Inserção, remoção, consulta e cálculo do tamanho;
- Comparando implementações com e sem **nó descritor**;
- Entendendo o impacto de ponteiros como **início**, **final** e **qtd** no custo das operações.

Análise de Complexidade nas Estruturas de Dados

Lista simplesmente encadeada (ponteiro-para-ponteiro)

- cria_lista →
- libera_lista →
- lista_vazia →
- tamanho_lista →
- insere_inicio →
- insere_final →
- remove_inicio →
- remove_final →

Análise de Complexidade nas Estruturas de Dados

Lista simplesmente encadeada (ponteiro-para-ponteiro)

- cria_lista → **O(1)**
- libera_lista → **O(n)** (percorre e libera todos os nós)
- lista_vazia → **O(1)** (testa ponteiro)
- tamanho_lista → **O(n)** (conta percorrendo)
- insere_inicio → **O(1)**
- insere_final → **O(n)** (precisa achar o último)
- remove_inicio → **O(1)**
- remove_final → **O(n)** (precisa do penúltimo)

Análise de Complexidade nas Estruturas de Dados

Lista simplesmente encadeada (nó descritor com inicio e qtd)

- cria_lista →
- libera_lista →
- lista_vazia →
- tamanho_lista →
- insere_inicio →
- insere_final →
- remove_inicio →
- remove_final →

Análise de Complexidade nas Estruturas de Dados

Lista simplesmente encadeada (nó descritor com inicio e qtd)

- cria_lista → **O(1)**
- libera_lista → **O(n)**
- lista_vazia → **O(1)**
- tamanho_lista → **O(1)** (usa qtd)
- insere_inicio → **O(1)**
- insere_final → **O(n)** (descritor não guarda final)
- remove_inicio → **O(1)**
- remove_final → **O(n)**

Análise de Complexidade nas Estruturas de Dados

Lista duplamente encadeada (nó descritor com inicio, final, qtd)

- cria_lista →
- libera_lista →
- lista_vazia →
- tamanho_lista →
- insere_inicio →
- insere_final →
- remove_inicio →
- remove_final →

Análise de Complexidade nas Estruturas de Dados

Lista duplamente encadeada (nó descritor com inicio, final, qtd)

- cria_lista → **O(1)**
- libera_lista → **O(n)**
- lista_vazia → **O(1)**
- tamanho_lista → **O(1)** (usa qtd)
- insere_inicio → **O(1)**
- insere_final → **O(1)** (usa ponteiro final)
- remove_inicio → **O(1)**
- remove_final → **O(1)**

Análise de Complexidade nas Estruturas de Dados

Lista simplesmente encadeada circular (nó descritor com final e qtd)

- cria_lista →
- libera_lista →
- lista_vazia →
- tamanho_lista →
- insere_inicio →
- insere_final →
- remove_inicio →
- remove_final →

Análise de Complexidade nas Estruturas de Dados

Lista simplesmente encadeada circular (nó descritor com final e qtd)

- cria_lista → **O(1)**
- libera_lista → **O(n)**
- lista_vazia → **O(1)**
- tamanho_lista → **O(1)** (usa qtd)
- insere_inicio → **O(1)** (insere após final)
- insere_final → **O(1)** (insere após final e atualiza final)
- remove_inicio → **O(1)** (ajusta final->prox)
- remove_final → **O(n)** (precisa achar o anterior ao final)

Análise de Complexidade nas Estruturas de Dados

Fila encadeada (nó descritor com inicio, final, qtd)

- cria_Fila →
- libera_Fila →
- insere_Fila (enqueue) →
- remove_Fila (dequeue) →
- consulta_Fila (peek) →
- Fila_vazia →
- tamanho_Fila →

Análise de Complexidade nas Estruturas de Dados

Fila encadeada (nó descritor com inicio, final, qtd)

- cria_Fila → **O(1)**
- libera_Fila → **O(n)**
- insere_Fila (enqueue) → **O(1)** (usa final)
- remove_Fila (dequeue) → **O(1)** (remove de inicio)
- consulta_Fila (peek) → **O(1)**
- Fila_vazia → **O(1)**
- tamanho_Fila → **O(1)** (usa qtd)

Análise de Complexidade nas Estruturas de Dados

Pilha encadeada (nó descritor com topo, qtd)

- cria_Pilha →
- libera_Pilha →
- insere_Pilha (push) →
- remove_Pilha (pop) →
- acessa_topo_Pilha (peek) →
- Pilha_vazia →
- tamanho_Pilha →

Análise de Complexidade nas Estruturas de Dados

Pilha encadeada (nó descritor com topo, qtd)

- cria_Pilha → **O(1)**
- libera_Pilha → **O(n)**
- insere_Pilha (push) → **O(1)**
- remove_Pilha (pop) → **O(1)**
- acessa_topo_Pilha (peek) → **O(1)**
- Pilha_vazia → **O(1)**
- tamanho_Pilha → **O(1)** (usa qtd)

Análise de Complexidade nas Estruturas de Dados

Funções Adicionais

Lista simplesmente encadeada

- imprime_lista → **O(n)**
- imprime_lista_reverso → **O(n)** tempo e **O(n)** espaço
- busca_valor → **O(n)**
- consulta_lista_posicao → **O(n)**

Análise de Complexidade nas Estruturas de Dados

Funções Adicionais

Lista duplamente encadeada

- imprime_lista → **O(n)**
- imprime_lista_reverso → **O(n)**
- busca_valor → **O(n)**
- consulta_lista_posicao → **O(min(pos, n−pos))**

Análise de Complexidade nas Estruturas de Dados

Funções Adicionais

Lista circular simplesmente encadeada

- imprime_lista → **O(n)**
- imprime_lista_reverso → **O(n)** tempo e **O(n)** espaço
- busca_valor → **O(n)**
- consulta_lista_posicao → **O(n)**

Estrutura de Dados Lineares

Introdução à Análise da Complexidade de Algoritmos